

SOFTWARE REQUIREMENTS SPECIFICATION

for

PayEase - Instalment Shop Management System

Prepared For: Academic Project Evaluation & Submission

Document Version: 1.0.0 (Production Release)

Date of Submission: 24-July-2026

Status: Completed and Fully Verified

Software Requirements Specification (SRS)

Project Name: PayEase - Instalment Shop Management System

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Project Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation

3. System Features

- 3.1 Authentication & User Management Module
- 3.2 Customer Management Module
- 3.3 Product & Inventory Management Module
- 3.4 Sales & Billing (POS) Module
- 3.5 Instalment Plan Management Module
- 3.6 Payment Tracking & Collection Module
- 3.7 Reports & Business Analytics Module
- 3.8 Settings & Backups Module

4. External Interface Requirements

- 4.1 User Interfaces
- 4.2 Software Interfaces
- 4.3 Communications Interfaces

5. Other Non-Functional Requirements

- 5.1 Performance Requirements
- 5.2 Safety & Security Requirements
- 5.3 Software Quality Attributes

6. Database Schema & UML Diagrams

- 6.1 Entity-Relationship (ER) Diagram
- 6.2 Use Case Diagram
- 6.3 Activity Diagrams
- 6.4 Class Diagram

1. Introduction

1.1 Purpose

The purpose of this document is to define the Software Requirements Specification (SRS) for the **PayEase - Instalment Shop Management System**. It specifies the functional and non-functional requirements of the system, acting as a structural blueprint for developers, test engineers, and administrators.

1.2 Document Conventions

This document follows the standard IEEE 830 template guidelines. Font styles, bold attributes, and syntax blocks are aligned with standard technical specifications.

1.3 Intended Audience and Reading Suggestions

This document is prepared for college project evaluators, developers, system administrators, and quality assurance testers.

- ****Evaluators****: Read Chapters 1 and 2 for context, then review the database and UML designs in Chapter 6.
- ****Developers****: Review Chapter 3 for detailed functional parameters and API designs.
- ****Admins****: Review Settings and Database Backups specifications in Chapter 3.8.

1.4 Project Scope

PayEase is a production-ready Web ERP system tailored for small to medium retail businesses offering products on credit and instalments. It automates credit checkouts, monthly EMI calculations, collections tracking, double-entry customer ledgers, and cash receipts printing.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Python Flask Framework Documentation (v3.0.3).
- SQLite3 Database File Format Specifications.

2. Overall Description

2.1 Product Perspective

PayEase is a standalone, browser-accessible ERP system. It features a responsive UI with dark/light mode toggles, interactive Chart.js widgets, and transactional database integrity.

2.2 Product Functions

- **Authentication**: Hashed password sign-ins, role restrictions (Super Admin, Admin, Staff).
- **Customer Ledgers**: Records customer info and double-entry debit/credit ledger lines.
- **POS Checkout**: Cart validations, down-payment hooks, and automatic instalment schedule generation.
- **Payment Collection**: Tracking exact/partial collections and advance cascading payments.
- **Data Exports**: Real-time PDF receipt generation and CSV/Excel exports.
- **Backups Manager**: Transaction-safe database backup, download, and restoration.

2.3 User Classes and Characteristics

1. **Staff (Cashiers)**: Perform basic operational tasks (registering customers, POS checkout, payments collection, printing receipts).
2. **Admin (Shop Owners)**: Full access to operational parameters, manual backup generation, and visual reports dashboards.
3. **Super Admin (Root Operators)**: Full system control, user account additions/deletions, settings configurations, and database restores.

2.4 Operating Environment

- **Platform**: Cross-platform (Windows Server, Linux/Ubuntu, macOS).
- **Runtime**: Python 3.10 / 3.11 environment.
- **Client**: Modern web browsers (Chrome, Edge, Firefox, Safari).

2.5 Design and Implementation Constraints

- **Type**: SQL-relational database.
- **Security**: Hashed passwords, CSRF tokens, session timeouts, and input form validations.
- **Compatibility**: Windows Unicode character support for standard outputs.

2.6 User Documentation

Includes:

- [User Manual](file:///c:/Instalmentshopmanage/docs/user_manual.md)
- [Administrator Manual](file:///c:/Instalmentshopmanage/docs/admin_manual.md)
- [Installation Guide](file:///c:/Instalmentshopmanage/docs/installation.md)
- [Deployment Guide](file:///c:/Instalmentshopmanage/docs/deployment.md)

3. System Features

3.1 Authentication & User Management Module

Allows system operators to log in and protects data files from unauthorized access.

- **Inputs**: Username/Email, Password.
- **Validation**: Hashed checks using `bcrypt`.
- **Outputs**: Redirects to dashboard or displays validation error.

3.2 Customer Management Module

Maintains customer directory and double-entry ledger lines.

- **Features**: Create Customer, Edit Profile, View Ledger Statements.
- **Calculation**: Outstanding balance computed as the sum of all active instalment plans dues.

3.3 Product & Inventory Management Module

Manages product listings, barcodes, category classifications, and current stock.

- **Alerts**: Prompts "LOW STOCK" alerts when current stock $\leq$ minimum threshold.
- **Logs**: Records an `InventoryMovement` entry for every stock modification.

3.4 Sales & Billing (POS) Module

Allows cashiers to process cart checkouts, calculate 18% GST tax, subtract stock, and spawn instalment plans.

- **Down Payment**: Deducts from total checkout cost.
- **Credit Sale**: Creates an `InstalmentPlan` if remaining balance > 0.

3.5 Instalment Plan Management Module

Maintains instalment schedules, due dates, monthly EMI calculations, and overdue checkers.

- **Schedules**: Spawns 6 monthly EMI rows.
- **Rescheduling**: Super Admin can shift individual due dates, cascading downstream schedules.

3.6 Payment Tracking & Collection Module

Records cash/card/UPI collections.

- **Exact EMI**: Marks target schedule row as `paid`.
- **Partial Collection**: Marks schedule row as `partially\_paid` and updates remaining balance.
- **Advance Collection**: Settles target schedule and cascades excess cash over to deduct from subsequent monthly schedules sequentially.
- **Ledger**: Inserts double-entry credit row reducing running customer balance.

3.7 Reports & Business Analytics Module

Compiles visual trends charts and CSV/Excel data streams.

- **Forecasting**: Runs linear trend regressions to project next month collections and revenue.
- **Role Boundary**: Staff accounts are blocked from financial reports and exports (returning 403 Forbidden).

3.8 Settings & Backups Module

Manages store profiles, prefixes, and backups.

- **Backups**: Safe SQLite transactional copy via `.backup()`.
- **Restore**: Super Admin only. Overwrites active `store.db` from a backup copy.

4. External Interface Requirements

4.1 User Interfaces

- Responsive Bootstrap 5 layouts.
- DataTables with sorting and instant searching.
- Chart.js visualization widgets.
- HSL color custom properties (Light/Dark themes).

4.2 Software Interfaces

- **Database**: SQLite3 (local development) and Supabase PostgreSQL (production).
- **WSGI Server**: Gunicorn.

4.3 Communications Interfaces

- HTTPS communication protocols.
- CSV and Excel file downloads.

5. Other Non-Functional Requirements

5.1 Performance Requirements

- Server response times < 200ms.
- Seeding and transaction operations complete in < 1s.
- Supports 1,600+ database records.

5.2 Safety & Security Requirements

- CSRF protections on forms.
- Role-based access control (RBAC).
- Input validations preventing SQL injections and XSS.
- Session timeouts on idle logins.

5.3 Software Quality Attributes

- **Maintainability**: Modular Flask blueprints.
- **Portability**: Containerized deployment support (Docker).
- **Reliability**: Transactional rollbacks on database operation failures.

6. Database Schema & UML Diagrams

Refer to the following diagrams in the project workspace:

- **ER Diagram**: [Database Schema](file:///c:/Instalmentshopmanage/docs/database.md)
- **Use Case Diagram**: [Use Case Diagram](file:///c:/Instalmentshopmanage/Diagrams/UseCaseOG.pdf)
- **Activity Diagrams**: [Activity Flowsheets](file:///c:/Instalmentshopmanage/docs/activity_diagrams.md)
- **Class Diagram**: [Class Diagram](file:///c:/Instalmentshopmanage/Diagrams/classdiagram.pdf)
- **Sequence Diagram**: [Sequence Diagram](file:///c:/Instalmentshopmanage/Diagrams/sequencediagram.pdf)